

Homework 2

Due: 5:59pm ET Thursday September 17

*This problem can be done in teams of up to 2 students.***Problem 1 Derivation of logistic regression gradient [10 points]**

As explained in class, logistic regression predicts a probability $\hat{y} \in (0, 1)$ for a binary label using the sigmoid (logistic) function:

$$\begin{aligned}\hat{y} &= \sigma(z) = \frac{1}{1 + \exp(-z)} \\ z &= \mathbf{x}^\top \mathbf{w} + b\end{aligned}\tag{1.1}$$

The cross-entropy cost function for a single observation $(\mathbf{x}, y)$ is:

$$f_{\text{CE}}(\mathbf{w}, b) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

- (6 pts) In this homework problem, you must complete the following derivation that is outlined below:

Derivation:

$$\begin{aligned}\nabla_{\mathbf{w}} f_{\text{CE}}(\mathbf{w}, b) &= \text{complete me...} \\ &= -\mathbf{x} (y - \hat{y})\end{aligned}$$

Hint: It is useful to first show that $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, i.e., $\nabla_{\mathbf{w}} \hat{y} = \mathbf{x} \hat{y}(1 - \hat{y})$.

- (2 pts) Now, show that

$$\nabla_b f_{\text{CE}}(\mathbf{w}, b) = -(y - \hat{y})$$

- (2 pts) Generalize the results above to a batch of n observations $(\mathbf{x}^{(i)}, y^{(i)})$, $i = 1, \dots, n$, using the superscript notation $\mathbf{x}^{(i)}$, $y^{(i)}$, $\hat{y}^{(i)}$. **No additional derivation is needed** – simply rewrite $\nabla_{\mathbf{w}} f_{\text{CE}}(\mathbf{w}, b)$ and $\nabla_b f_{\text{CE}}(\mathbf{w}, b)$ using the appropriate notation.

Problem 2 Numpy implementation of softmax regression [20 points]

Train a 2-layer softmax neural network to classify images of fashion items (10 different classes, such as shoes, t-shirts, dresses, etc.) from the Fashion MNIST dataset. The input to the network will be a 28×28 -pixel image (converted into a 784-dimensional vector); the output will be a vector of 10 probabilities (one for each class). The cross-entropy loss function that you minimize should be

$$f_{\text{CE}}(\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(10)}, b^{(1)}, \dots, b^{(10)}) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^{10} y_k^{(i)} \log \hat{y}_k^{(i)}$$

where n is the number of examples and α is a regularization constant.. Note that each $\hat{y}_k$ implicitly depends on all the weights $\mathbf{W} = [\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(10)}]$ and biases $\mathbf{b} = [b^{(1)}, \dots, b^{(10)}]$. The gradients of f_{CE} that you will need for gradient descent are given by:

$$\nabla_{\mathbf{w}^{(k)}} f_{\text{CE}} = -\frac{1}{n} \sum_{i=1}^n \mathbf{x}^{(i)} \left(y_k^{(i)} - \hat{y}_k^{(i)} \right), \quad \nabla_{b_k} f_{\text{CE}} = -\frac{1}{n} \sum_{i=1}^n \left(y_k^{(i)} - \hat{y}_k^{(i)} \right)$$

for each class $k = 1, \dots, 10$.

To get started, first download the Fashion MNIST dataset from the following web links:

- https://s3.amazonaws.com/jrwprojects/fashion_mnist_train_images.npy
- https://s3.amazonaws.com/jrwprojects/fashion_mnist_train_labels.npy
- https://s3.amazonaws.com/jrwprojects/fashion_mnist_test_images.npy
- https://s3.amazonaws.com/jrwprojects/fashion_mnist_test_labels.npy

To save some time, you can use this STARTER CODE: <https://canvas.wpi.edu/courses/87875/files/folder/assignments?preview=8763681>.

These files can be loaded into numpy using `np.load`. Each “labels” file consists of a 1-d array containing n labels (valued 0-9), and each “images” file contains a 2-d array of size $n \times 784$, where n is the number of images. **Hint:** Images are in grayscale and represented by values between 0 and 255. Mapping the inputs to the interval $[-0.5, 0.5]$ will help train the neural network.

Next, implement stochastic gradient descent (SGD) to minimize the cross-entropy loss function on this dataset. Regularize the weights but *not* the biases. Optimize the hyperparameters (e.g., learning rate, regularization strength), **considering at least 9 combinations of hyperparameter values**. You should also use the same methodology as for the previous homework, including the splitting of the training files into validation and training portions.

Performance evaluation: Once you have tuned the hyperparameters and optimized the weights so as to maximize performance on the validation set, then: (1) **stop** training the network and (2) evaluate the network on the **test** set. Record the performance both in terms of (unregularized) cross-entropy loss (smaller is better) and percent correctly classified examples (larger is better); put this information into the PDF you submit.

Problem 3 Instrumenting and Tracking Experiments with Weights & Biases [15 points]

In Problem 2, you implemented softmax regression and manually tuned its hyperparameters by running your code multiple times with different settings. You likely discovered that this process can be challenging. How did you keep track of which learning rate and regularization strength produced which validation accuracy? Was it easy to compare the training dynamics of run #3 with run #8? How could you share your findings with a collaborator to defend your choice of the final model?

In academic research and industry, this process of tracking, comparing, and managing experiments is a core component of Machine Learning Operations (MLOps). It is managed using specialized platforms designed to make machine learning a systematic and reproducible science. In this problem, you will augment your solution from Problem 2 using one of the most popular experiment tracking platforms: Weights & Biases (W&B). Your goal is not just to run experiments, but to log them methodically and use the W&B dashboard to analyze your results in a way that was not possible with your previous manual approach.

This problem is based on the steps described in this tutorial.

Part A: Setup and W&B Integration [5 points]

Your first task is to set up your environment and instrument your existing script from Problem 2 to communicate with the W&B platform.

1. Create a free personal account at <https://wandb.ai/site>.
2. Install the W&B library in your Python environment using a package manager: `pip install wandb`.
3. (Recommended, but optional step) Authenticate your machine by running the following command in your terminal: `wandb login`. You will be prompted to paste an API key, which you can find in the "Settings" section of your W&B profile. **Note:** if you don't log in using the terminal, you will be prompted to insert your API key when first running your code.
4. Copy your script from Problem 2 to create a modified version. At the beginning of your new script, after your imports, add `import wandb`.
5. (1 point) Structure your code to execute the 9+ hyperparameter combinations from Problem 2 within a loop. For each unique combination of hyperparameters, you will perform the steps below to initialize and track a distinct experiment run.
6. (1 point) Inside this loop, before you begin training, use the `wandb.init()` function to start a new run. This function is the entry point for all tracking. You must configure it as follows:
7. (1 point) Set the `project` argument to a single, descriptive name for all runs in this assignment, for example, "cs541-hw1-fashion-mnist". All your runs will be grouped under this project on your dashboard.
8. (1 point) Pass a Python dictionary containing the hyperparameters for the *current run* to the `config` argument. This is crucial for telling W&B which settings correspond to which results. For example: `{'learning_rate': 0.01, 'alpha': 1e-4, 'epochs': 20}`.
9. To ensure your code uses the exact hyperparameters logged by W&B, it is a best practice to access them throughout your training logic via the run's config object. For example, after initialization, use `learning_rate = wandb.config.learning_rate` instead of your original variable. This establishes the W&B configuration as the single source of truth for the experiment, a key principle of reproducibility.
10. (1 point) Use the `tags` (e.g., `tags=['lr_sweep']`) or `group` argument of `wandb.init()` to organize the runs from this assignment into a meaningful set. In 1-2 sentences, explain how this organization would help a collaborator understand or reproduce your experiments.

Part B: Logging Metrics During Training [4 points]

With initialization in place, you must now log the model's performance metrics to W&B throughout the training process. This allows you to capture the dynamics of learning, not just the final result.

1. Locate the training loop (the one that iterates over epochs) in your script.
2. Within this loop, immediately after you evaluate your model on the validation set at the end of an epoch, add a call to the `wandb.log()` function.
3. As described in the W&B documentation, this function accepts a dictionary of metrics. Your dictionary must contain, at a minimum, the validation loss and validation accuracy for the current epoch. It is also good practice to log the epoch number itself.

4. (1 point) A correct log call should look similar to this: `wandb.log({'epoch': epoch, 'val_loss': validation_loss, 'val_accuracy': validation_accuracy})`.
5. (1 point) Ensure this line is executed for every epoch of every hyperparameter combination run. W&B will automatically aggregate this data to generate time-series plots for each metric, allowing you to visualize learning curves for every experiment.
6. (1 point) In addition to the validation metrics, also compute and log the *training* loss and training accuracy for the current epoch (evaluated on the training set, or a fixed subsample of it for speed), e.g.: `wandb.log({'epoch': epoch, 'train_loss': train_loss, 'train_accuracy': train_accuracy, 'val_loss': val_loss, 'val_accuracy': val_accuracy})`.
7. (1 point) For your best-performing run, take a screenshot of the W&B line plot overlaying training and validation loss (or accuracy) vs. epoch, and include it in your PDF report. In 1-2 sentences, state whether the curves indicate overfitting, underfitting, or a good fit.

Part C: Analysis and Visualization [6 points]

After running all your hyperparameter experiments and logging them to W&B, the final and most important step is to use the platform's tools to analyze your results.

1. Navigate to your project page on the W&B website. Under the tab “Runs”, you should see a table summarizing all the runs you executed, with columns for each of your configured hyperparameters and logged metrics.
2. Your task is to analyze the relationship between your chosen hyperparameters and the model's performance. In the W&B user interface, create a Parallel Coordinates Plot. This plot is a powerful tool for visualizing how different hyperparameter values correlate with output metrics. Configure the plot to show your hyperparameters (e.g., `learning_rate`, `alpha`) as axes, along with your primary performance metric (e.g., the final `val_accuracy`).
3. (1 point) Take a screenshot of your configured Parallel Coordinates Plot and include it in your submitted PDF report.
4. (3 points) Below the screenshot, write a brief analysis (2-3 paragraphs) that answers the following questions:
 - Based on the visualization, which hyperparameter had the most significant impact on validation accuracy? How does the plot make this apparent?
 - What range of values for each hyperparameter appears to be most promising for achieving high performance?
 - How does this visualization-driven analysis make it easier to draw these conclusions compared to looking at a simple table of numbers or your console output from Problem 2? What are the key advantages of this systematic approach?
5. (2 points) Using the sortable “Runs” table on your W&B project page, identify the single run with the highest validation accuracy. Report its exact hyperparameter values (learning rate, α , batch size, etc.) and, after evaluating that configuration on the **test** set (as instructed in Problem 2), report the resulting test cross-entropy loss and test accuracy.

Problem 4 Implementing gradient computation via the multivariate chain rule (linear case) [15 points]

Consider the two vector fields below taken from the Supplemental Materials on Jacobian matrices and multivariate chain rule, slide 9:

$$f\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = x_1 - 2x_2 + x_3/4 \quad g\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} x_1 + 2x_2 \\ x_2 \\ -x_1 + 3 \end{bmatrix}.$$

Let the initial input to g be $\mathbf{x} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$.

Note: To keep things simple, we disregard activation functions in this problem – f and g are treated purely as affine transformations, and their composition $(f \circ g)$ is a composition of affine transformations (i.e., what you'd get from a linear neural network with no nonlinearities).

1. Show how to represent each function as an affine transformation of the form $\mathbf{W}\mathbf{x} + \mathbf{b}$. For function g , denote the parameters by $\mathbf{W}_1$ and $\mathbf{b}_1$, whereas for function f , denote the parameters by $\mathbf{W}_2$ and $\mathbf{b}_2$. [1 point]
2. Implement a python function `AffineTransformation(W, b, x)` that can be used to compute either function by taking as input two arrays of parameters $\mathbf{W}$ and $\mathbf{b}$ and a vector $\mathbf{x}$. [1 point]
3. Building on the previous function, implement a new function `Composition(L, x)` that calculates the composition of affine transformations represented as a list of pairs $L = [(\mathbf{W}_1, \mathbf{b}_1), \dots, (\mathbf{W}_n, \mathbf{b}_n)]$ and an initial vector x . **Use the following convention:** the functions are applied from the smallest to the largest index. **Return all the activation values (i.e., intermediate results) as a list variable $\mathbf{z} = (z_1, \dots, z_n)$.** [2 points]
4. How would you call the previous function to compute $(f \circ g)(\mathbf{x})$, for $\mathbf{x} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$?
Return ONLY the final output. [1 point]
5. Given an affine transformation $\mathbf{z}_2 = f(\mathbf{z}_1)$ parameterized by $\mathbf{W}_2$ and $\mathbf{b}_2$, derive [2 points]:

$$\frac{\partial \mathbf{z}_2}{\partial \text{vec}[\mathbf{W}_2]}(\mathbf{z}_1) \quad \text{and} \quad \frac{\partial \mathbf{z}_2}{\partial \mathbf{b}_2}(\mathbf{z}_1).$$

6. Write equations for: [1 point]

$$\frac{\partial (f \circ g)}{\partial \mathbf{z}_1}(\mathbf{x})$$

7. For some scalar function $y = (f \circ g)(\mathbf{x})$, where $\mathbf{z} = g(\mathbf{x})$ is an affine transformation parameterized by $\mathbf{W}_{n \times m}$ and $\mathbf{b}_{n \times 1}$, we know that

$$\nabla_{\mathbf{W}} y = \text{unvec} \left[\frac{\partial y}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \text{vec}[\mathbf{W}]} \right]$$

Last, if $\frac{\partial y}{\partial \mathbf{z}} = \mathbf{p}^\top$, then

$$\nabla_{\mathbf{W}} y = \mathbf{p} \mathbf{x}^\top.$$

Using the multivariate chain rule, write equations for: [2 points]

$$\frac{\partial(f \circ g)}{\partial \mathbf{b}_1} \quad \text{and} \quad \nabla_{\mathbf{W}_1}(f \circ g)(\mathbf{x})$$

8. Using the list of parameters L , the input vector $\mathbf{x}$ and all the activation values $\mathbf{z}$ obtained from the `Composition`, implement a function `ComputeGradients(L, x, z)` that returns the gradients of z_n (a scalar) for all the parameters in L , i.e., $\nabla_{b_i} z_n = (\partial z_n / \partial [\mathbf{b}_i])^\top$. and $\nabla_{\mathbf{W}_i} z_n$. **To earn full marks, the solution should apply to any $n \geq 2$.** [5 points]

Submission

Submit one PDF file that includes your notes for the theoretical problems (scanned or typed) and screenshots of your code for the programming problems. All material in the submitted PDF must be presented in a clear and readable format.

Teamwork

You may complete this homework assignment either individually or in teams up to 2 people.

If you are working as part of a group, then indicate the members your submission.